how would I make it load said ratings

Wednesday 3:44 PM
Worked for 7s

For your Drafter app, put a file picker and a **Load Ratings** button on the home screen. When you choose `ratings.csv` and click the button, Drafter passes the file's contents to the route as a string. Drafter FileUpload docs

1. Add `ratings: dict[str, list[int]]` to your `State` dataclass and pass `ratings={}` when you create your starting state. Each food name maps to its list of scores.

2. Add these two items to the `Page` content list in `index`:

```
None
FileUpload("ratings_file", accept=".csv"),
Button("Load Ratings", "load_ratings"),
```

3. Add `import csv` and `from io import StringIO` at the top of your program. Then put this route above `start_server(...)`:

```
None
@route
def load_ratings(state: State, ratings_file: str | None) -> Page:
    if not ratings_file:
        return Page(state, [
            "Choose your ratings CSV first.",
            Button("Back to Home", "index"),
        ])

    loaded_ratings = {}
    try:
        for food_name, score in csv.reader(StringIO(ratings_file)):
            loaded_ratings.setdefault(food_name,
[]).append(int(score))
    except (ValueError, csv.Error):
        return Page(state, [
```

```
            "Each row should contain a food name and a whole-number
rating.",
            Button("Back to Home", "index"),
        ])

    state.ratings = loaded_ratings
    return index(state)
```

This expects the same format as the earlier `save_rating` function, with one food name and score per row and no header. Importing replaces the ratings in memory, so loading the same file twice won't double-count them. In your search route, `state.ratings.get(food.name, [])` gives you the scores for the selected food.

Wednesday 3:44 PM

my ratings system does not survive a restart, what can i add to fix this

10:04 AM
Worked for 9s

Add two things to your Drafter app: **save the ratings when someone submits a score**, and **load them when the app starts**. `State` starts fresh on a restart, so use the browser's `localStorage` to keep a copy automatically. This example assumes you're using Drafter's Pyodide engine and the `dict[str, list[int]]` ratings field from earlier. How Drafter works · Pyodide browser access

Add this above `start_server(...)`:

None
```
import json

RATINGS_KEY = "menu-lookup-ratings"

def load_saved_ratings() -> dict[str, list[int]]:
    try:
        from js import localStorage
    except ImportError:
        # Drafter also runs startup code in regular Python.
```

```
        return {}

    saved = localStorage.getItem(RATINGS_KEY)
    return json.loads(saved) if saved else {}

def save_ratings(ratings: dict[str, list[int]]) -> None:
    from js import localStorage

    localStorage.setItem(RATINGS_KEY, json.dumps(ratings))
```

In your rating route, call `save_ratings` **right after** you add the score and before you return the page:

```
None
state.ratings.setdefault(food_name, []).append(score)
save_ratings(state.ratings)
```

In your `start_server(State(...))` call, initialize `ratings` with `ratings=load_saved_ratings()` instead of `ratings={}`.

To test it, submit a rating, restart the app, and open the same browser address, including the same port. The rating should still be there. If you already have ratings in a CSV, load them with your existing import button and call `save_ratings(state.ratings)` after a successful import to keep those too.

10:04 AM
from drafter import * from dataclasses import dataclass import csv '"LLM Writt…

current

10:05 AM
Worked for 8s

Your current code already calls `load_ratings()` at startup. The problem is where `save_rating()` writes: when you click **Save Rating**, Drafter runs `rating_entered` in the browser, so `open("ratings.csv", "a")` writes to the browser's temporary filesystem. That explains why the rating can show up immediately and disappear after a restart. [How Drafter works]

Keep your `Ratings` dataclass and `State.Rating` field. If your app uses Drafter's Pyodide engine, replace your two CSV functions with these and put them **below** the `Ratings` dataclass:

```
None
import json

RATINGS_KEY = "menu-lookup-ratings"

def load_ratings() -> list[Ratings]:
    try:
        from js import localStorage
    except ImportError:
        # Drafter also runs startup code in regular Python.
        return []

    saved = localStorage.getItem(RATINGS_KEY)
    if not saved:
        return []

    return [
        Ratings(food_name, scores)
        for food_name, scores in json.loads(saved).items()
    ]

def save_rating(food_name: str, rating: int) -> None:
    from js import localStorage

    saved = localStorage.getItem(RATINGS_KEY)
    scores_by_food = json.loads(saved) if saved else {}
    scores_by_food.setdefault(food_name, []).append(rating)
    localStorage.setItem(RATINGS_KEY, json.dumps(scores_by_food))
```

These use the same function names and return types your code already expects. `rating_entered` saves the new score and reloads `state.Rating`, and `start_server(State("", 0.0, False, load_ratings()))` loads the scores on startup. [Pyodide browser access]

Submit a rating, restart the app, and search for that food again using the same browser and address, including the port. This saves new ratings in browser storage. If you have an older CSV on your computer, it needs a one-time import.

10:05 AM

what is json

10:12 AM

JSON stands for **JavaScript Object Notation**. It's a text format for storing data. In your app, the saved ratings might look like this:

```
None
{"Thai Basil Curry Stir Fry": [4, 5]}
```

The food name is the key, and `[4, 5]` is the list of scores. Python's built-in `json` module handles the conversion:

- `json.dumps(scores_by_food)` turns your Python dictionary into text that the browser can save.
- `json.loads(saved)` turns that text back into a Python dictionary so your app can use the scores again.

In your code, `localStorage` keeps the text between runs, and JSON gives that text a structure Python can read back.

10:12 AM

how to change background color and text color

10:44 AM
Worked for 9s

To change the background and text colors across your Drafter app, add this **just above your existing `start_server(...)` line**, outside any route. Drafter custom CSS docs

```
None
add_website_css("""
    body {
        background-color: lightblue;
        color: midnightblue;
```

```
    }
""")
```

`background-color` controls the page background. `color` controls the default text color. You can use color names like `white` or hex codes like `#f4f7fb`.

To give a single heading its own color, add `style_color` directly to it. Drafter styling docs

None
```
Header("Menu Lookup", style_color="darkgreen")
```

10:45 AM

how to make text bold

10:45 AM

In Drafter, add `style_font_weight="bold"` to a `Text` component in your `Page` list. Drafter styling docs

None
```
Text("Know what you want?", style_font_weight="bold"),
```

In your `index` route, replace the plain `"Know what you want?"` string with that line.

10:45 AM
from drafter import * from dataclasses import dataclass import json RATINGS_KEY…

why is my time constantly reverting to 12am

1:13 PM
Worked for 10s

Your dropdown resets because `check_hours` calls `index(state, time)`, which builds a new dropdown. Your `SelectBox` doesn't use the `time` passed to `index`, so Drafter selects the first option again, `"12 AM"`. Drafter SelectBox docs

Change `index`'s default from `""` to `"12 AM"` and pass `time` as the dropdown's `default_value`. Here `time_options` stands for your existing list of 24 choices:

None

```
def index(state: State, time: str = "12 AM") -> Page:
    # Inside your Page content list:
    SelectBox("time", time_options, default_value=time,
on_change=check_hours)
```

That keeps the selected time when `check_hours` redraws the home page. If you also want to keep it after visiting another screen and clicking **Back to Home**, store the selection in `State` and use that field as the dropdown's default.

There's a separate bug in `check_hours`: your comparisons treat `12 AM` as open and `12 PM` as closed. Convert `12` to `0` before adding `12` for PM, then check the resulting 24-hour value against your opening and closing hours.